目標

想要先寫 WAF 的 rule 來阻擋 nuclei 的攻擊

- 先看 nuclei 會怎麼打目標
- 觀察完畢後針對攻擊頻率寫 rate limit rule

防禦

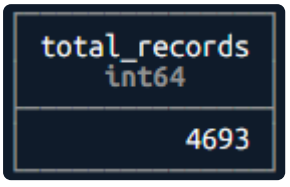
先針對這個 log.gz 來看，這是在沒有 WAF 的情況下，我用 nuclei 掃描 CVE，會出現 4693 筆 HTTP request

要用 duckdb

```
# 設定環境變數 (log 的 s3 url)
SET VARIABLE log_path = 's3://aws-waf-logs-
langflowcf/AWSLogs/227469555418/WAFLogs/cloudfront/CreatedByCloudFront-
f35d9cf1/2026/05/28/07/35/227469555418_waflogs_cloudfront_CreatedByCloudFront-
f35d9cf1_20260528T0735Z_41ebc8ec.log.gz';
```

```
# 看有多少筆資料
SELECT count(*) AS total_records
  FROM read_json_auto(getvariable('log_path'));
```

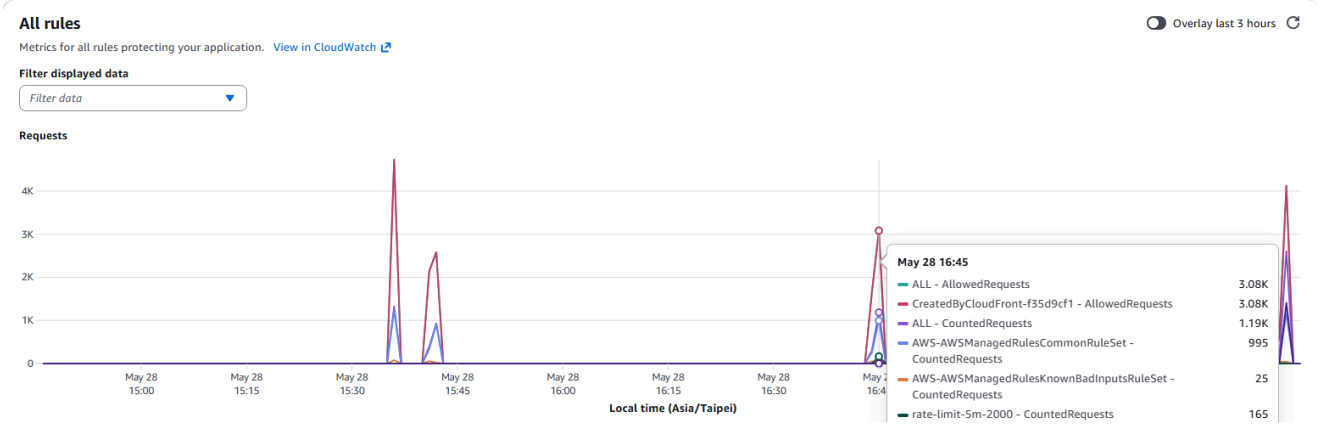
總共也 4693 筆



- 為了應付上述的這種掃描，決定直接加 rate limit 的 WAF rule

WAF Rule

一開始決定設成五分鐘 2000 次，但發現 nuclei 掃 CVE 的流量本來就有將近兩千筆會被其他 rule 擋下，所以我把 rate limit 縮到 5 分鐘 700 筆。

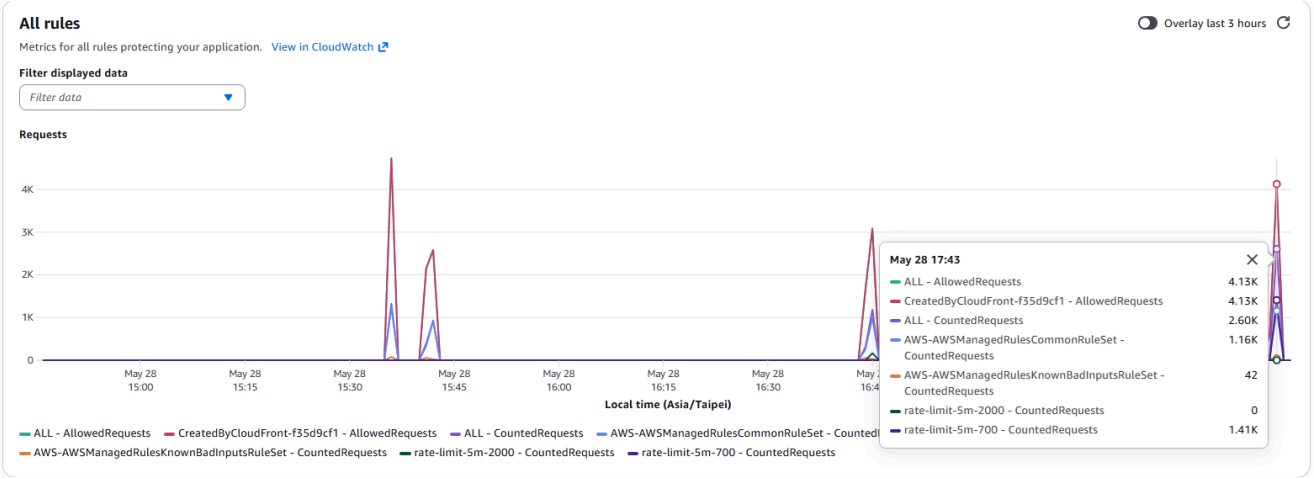


rate-limit-5m-700

```
{
    "Name": "rate-limit-5m-700",
    "Priority": 3,
```

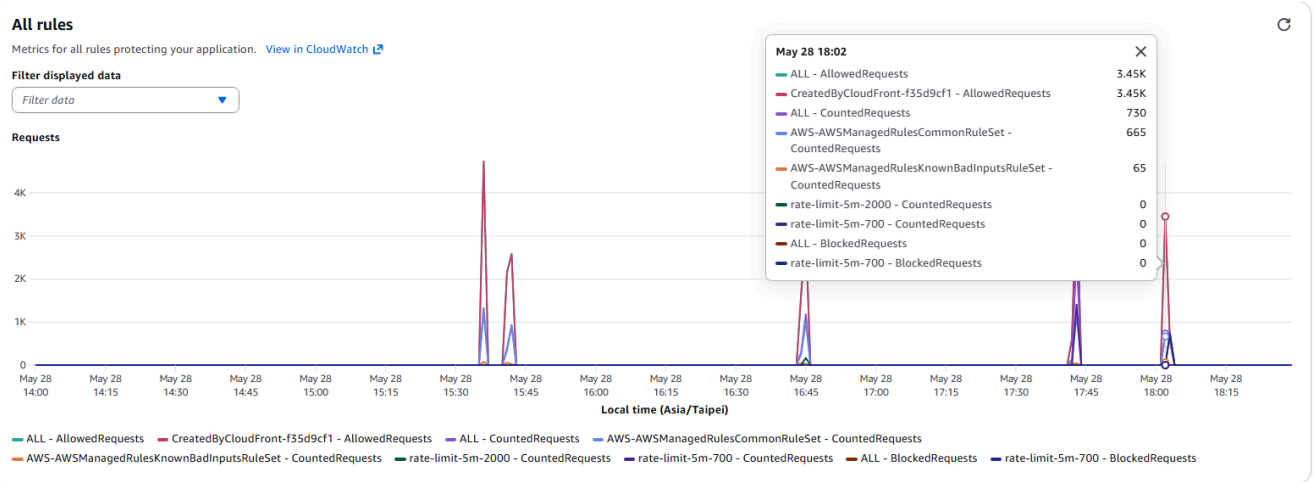
```
    "Statement": {
        "RateBasedStatement": {
            "Limit": 700,
            "AggregateKeyType": "IP",
            "EvaluationWindowSec": 300
        }
    },
    "VisibilityConfig": {
        "SampledRequestsEnabled": true,
        "CloudWatchMetricsEnabled": true,
        "MetricName": "rate-limit-5m-700"
    },
    "Action": {
        "Count": {}
    }
}
```

5 分鐘內限制 700 應該有改善，基本上就只要處理 700 筆資料就好，接下來會把這條 rule 從 count 改成 block



改成 block 之後發現 rule 失效了，決定把 log 下載下來分析

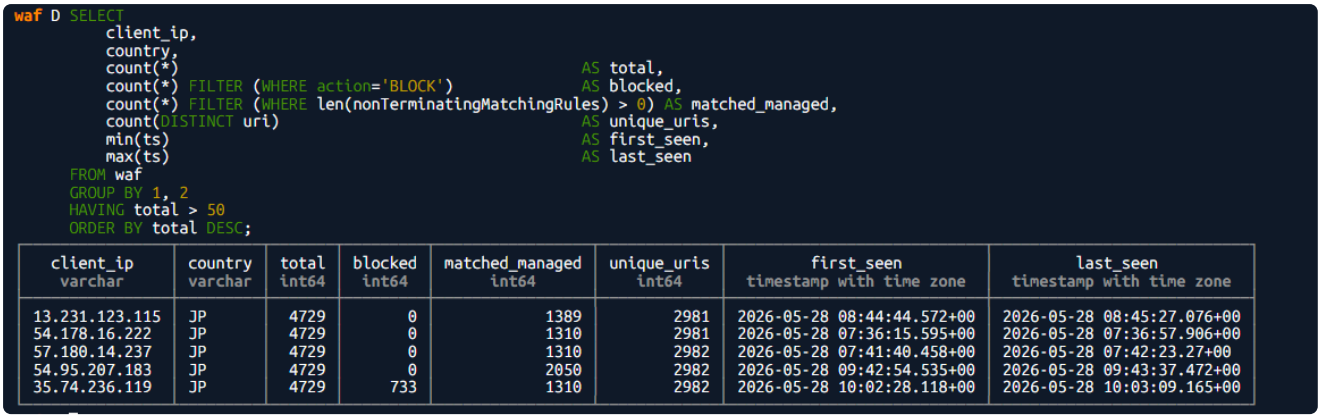
在圖片中可以看到通過了 3.45k 筆的 request 但實際上應該要在第 700 筆開始阻擋




```
duckdb -c "
  COPY (
    SELECT * FROM read_json_auto('s3://aws-waf-logs-
langflowcf/AWSLogs/227469555418/WAFLogs/cloudfront/CreatedByCloudFront-
f35d9cf1/2026/05/28/*/*/*.log.gz')
  ) TO 'waflog-20260528.parquet' (FORMAT PARQUET, COMPRESSION 'zstd');
  " ls -lh waflog-20260528.parquet
```

用 DuckDB 去看會發現其實它有 block，只是數量沒有這麼多，先推測可能是延遲?

```sql
SELECT
    client_ip,
    country,
    count(*)                                                AS requests,
    count(DISTINCT uri)                                     AS unique_uris,
    count(*) FILTER (WHERE len(nonTerminatingMatchingRules) > 0) AS matched_rules,
    round(count(DISTINCT uri) * 100.0 / count(*), 1)    AS uri_diversity_pct
FROM waf
GROUP BY 1, 2
HAVING requests > 30
ORDER BY matched_rules DESC, uri_diversity_pct DESC
LIMIT 20;
```



```
waf D SELECT
        client_ip,
        country,
        count(*)                                                AS total,
        count(*) FILTER (WHERE action='BLOCK')                  AS blocked,
        count(*) FILTER (WHERE len(nonTerminatingMatchingRules) > 0) AS matched_managed,
        count(DISTINCT uri)                                     AS unique_uris,
        min(ts)                                                 AS first_seen,
        max(ts)                                                 AS last_seen
    FROM waf
    GROUP BY 1, 2
    HAVING total > 50
    ORDER BY total DESC;
```

client_ip varchar	country varchar	total int64	blocked int64	matched_managed int64	unique_uris int64	first_seen timestamp with time zone	last_seen timestamp with time zone
13.231.123.115	JP	4729	0	1389	2981	2026-05-28 08:44:44.572+00	2026-05-28 08:45:27.076+00
54.178.16.222	JP	4729	0	1310	2981	2026-05-28 07:36:15.595+00	2026-05-28 07:36:57.906+00
57.180.14.237	JP	4729	0	1310	2982	2026-05-28 07:41:40.458+00	2026-05-28 07:42:23.27+00
54.95.207.183	JP	4729	0	2050	2982	2026-05-28 09:42:54.535+00	2026-05-28 09:43:37.472+00
35.74.236.119	JP	4729	733	1310	2982	2026-05-28 10:02:28.118+00	2026-05-28 10:03:09.165+00

詳細去看時間軸，發現 WAF 是在 10:03:04 (UTC) 才開始介入。原本應該在 10:02:43 就開始阻擋，跟實際差了大約 20 秒，我猜是 WAF 有延遲?去翻 AWS 的[文件](Rate-based rule caveats in AWS WAF - AWS WAF, AWS Firewall Manager, AWS Shield Advanced, and AWS Shield network security director) 它說可能會有延遲但不超過 30 秒。所以我覺得不該只用 IP 進行防禦，而是希望能在最短時間內觸發 block。對掃描工具而言，通常會爆破特定路徑或密碼，所以應該可以做到，只要有人在特定時間內對登入 endpoint、或 /admin、.git 這種隱藏路徑打太多次，就 block 掉

```sql
SELECT
    date_trunc('second', ts) AS sec,
    action,
    count(*) AS cnt
FROM waf
WHERE client_ip = '35.74.236.119'
GROUP BY 1, 2
ORDER BY 1, 2;
```

timestamp with time zone	varchar	int64
2026-05-28 10:02:28+00	ALLOW	76
2026-05-28 10:02:29+00	ALLOW	91
2026-05-28 10:02:30+00	ALLOW	101
2026-05-28 10:02:31+00	ALLOW	124
2026-05-28 10:02:32+00	ALLOW	115
2026-05-28 10:02:33+00	ALLOW	92
2026-05-28 10:02:34+00	ALLOW	117
2026-05-28 10:02:35+00	ALLOW	103
2026-05-28 10:02:36+00	ALLOW	121
2026-05-28 10:02:37+00	ALLOW	105
2026-05-28 10:02:38+00	ALLOW	103
2026-05-28 10:02:39+00	ALLOW	117
2026-05-28 10:02:40+00	ALLOW	115
2026-05-28 10:02:41+00	ALLOW	96
2026-05-28 10:02:42+00	ALLOW	126
2026-05-28 10:02:43+00	ALLOW	102
2026-05-28 10:02:44+00	ALLOW	117
2026-05-28 10:02:45+00	ALLOW	111
2026-05-28 10:02:46+00	ALLOW	106
2026-05-28 10:02:47+00	ALLOW	125
2026-05-28 10:02:48+00	ALLOW	140
2026-05-28 10:02:49+00	ALLOW	123
2026-05-28 10:02:50+00	ALLOW	122
2026-05-28 10:02:51+00	ALLOW	99
2026-05-28 10:02:52+00	ALLOW	101
2026-05-28 10:02:53+00	ALLOW	113
2026-05-28 10:02:54+00	ALLOW	58
2026-05-28 10:02:55+00	ALLOW	104
2026-05-28 10:02:56+00	ALLOW	107
2026-05-28 10:02:57+00	ALLOW	107
2026-05-28 10:02:58+00	ALLOW	106
2026-05-28 10:02:59+00	ALLOW	109
2026-05-28 10:03:00+00	ALLOW	109
2026-05-28 10:03:01+00	ALLOW	106
2026-05-28 10:03:02+00	ALLOW	119
2026-05-28 10:03:03+00	ALLOW	128
2026-05-28 10:03:04+00	ALLOW	82
2026-05-28 10:03:04+00	BLOCK	69
2026-05-28 10:03:05+00	BLOCK	143
2026-05-28 10:03:06+00	BLOCK	144
2026-05-28 10:03:07+00	BLOCK	147
2026-05-28 10:03:08+00	BLOCK	146
2026-05-28 10:03:09+00	BLOCK	84

43 rows 3 columns

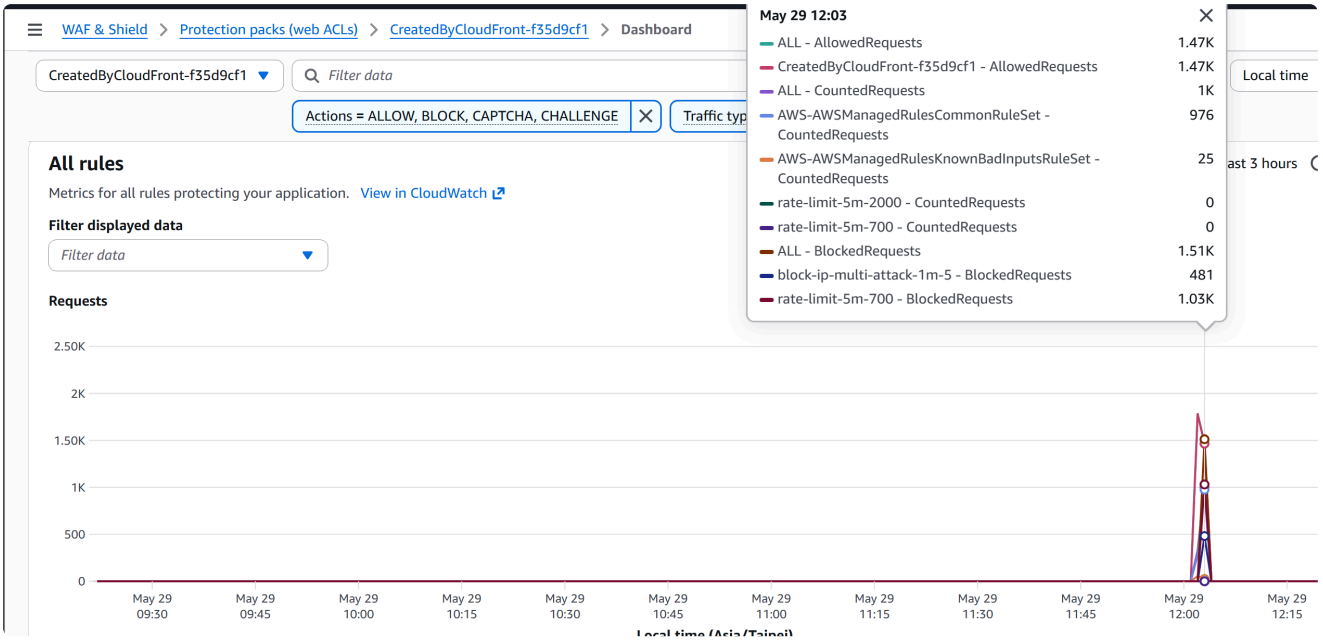
在寫 rule 的時候發現 AWS 內建的 rules 已經會阻擋一些特定路徑（但也就只是阻擋特殊路徑）。聯想到 rate limit，我想寫一條規則，若偵測到某個 IP 多次被 AWS WAF 現有的預設規則 match，就把這個 IP 的請求 block，而且解鎖時間是動態的。AWS 的 rate-based rule 邏輯是回頭看過去 n 秒內這個 IP 觸發規則的次數，所以攻擊者持續打就會持續被封鎖，使用者誤觸則會在短時間內自動恢復權限。（突然想到之前在戳別人的詐騙網站，戳到一半 IP 被封了，但我同學的電腦還是看得到網站，可能就是有類似的檢查）

rule

```
{
    "Name": "block-ip-multi-attack-1m-5",
    "Priority": 6,
    "Statement": {
        "RateBasedStatement": {
            "Limit": 10,
            "EvaluationWindowSec": 60,
            "AggregateKeyType": "IP",
            "ScopeDownStatement": {
                "OrStatement": {
                    "Statements": [
                        {
                            "LabelMatchStatement": {
                                "Scope": "NAMESPACE",
                                "Key": "awswaf:managed:aws:admin-protection:"
                            }
                        },
                        {
```

```
                        "LabelMatchStatement": {
                            "Scope": "NAMESPACE",
                            "Key": "awswaf:managed:aws:known-bad-inputs:"
                        }
                    },
                    {
                        "LabelMatchStatement": {
                            "Scope": "NAMESPACE",
                            "Key": "awswaf:managed:aws:core-rule-set:"
                        }
                    }
                ]
            }
        }
    },
    "Action": {"Block": {}},
    "VisibilityConfig": {
        "SampledRequestsEnabled": true,
        "CloudWatchMetricsEnabled": true,
        "MetricName": "block-ip-multi-attack-1m-5"
    }
}
```

驗收成果，真正擋住的只有 1.5 k 左右的流量，畢竟 waf 本來就會有差不多 20 秒左右的延遲



但如果換範圍大一點的 template 那效果會很可觀，總共 13 K 的 request 可以攔截 9 K 左右
今天我如果是攻擊者，應該不會希望我的大規模掃描被攔截這麼多

Action totals

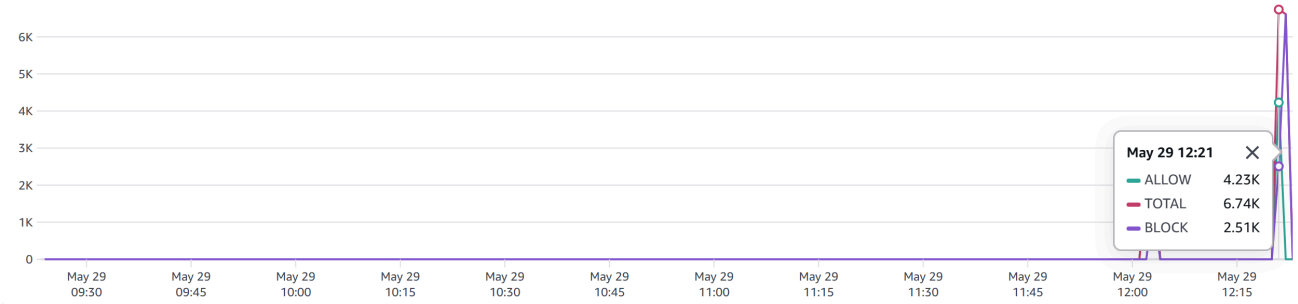
Overlay last 3 hours

Action totals for the specified time range. View in CloudWatch

Filter displayed data

Filter data

Requests



Action totals

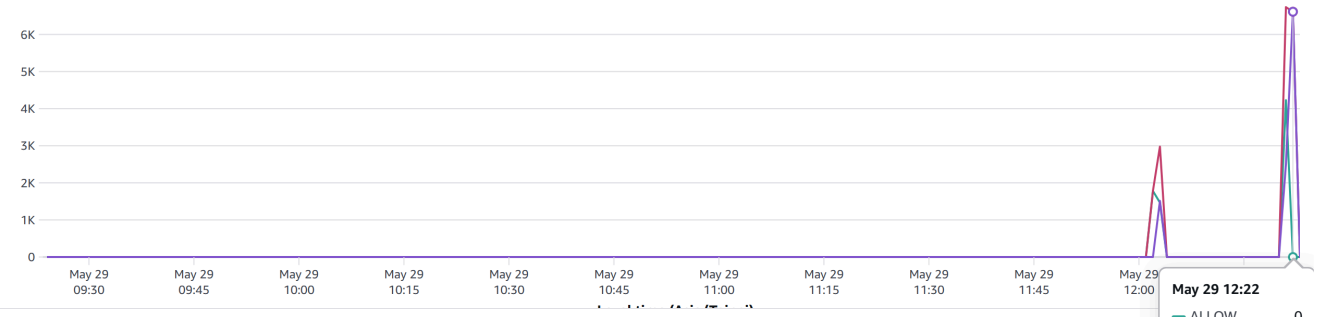
Overlay last 3 hours

Action totals for the specified time range. View in CloudWatch

Filter displayed data

Filter data

Requests

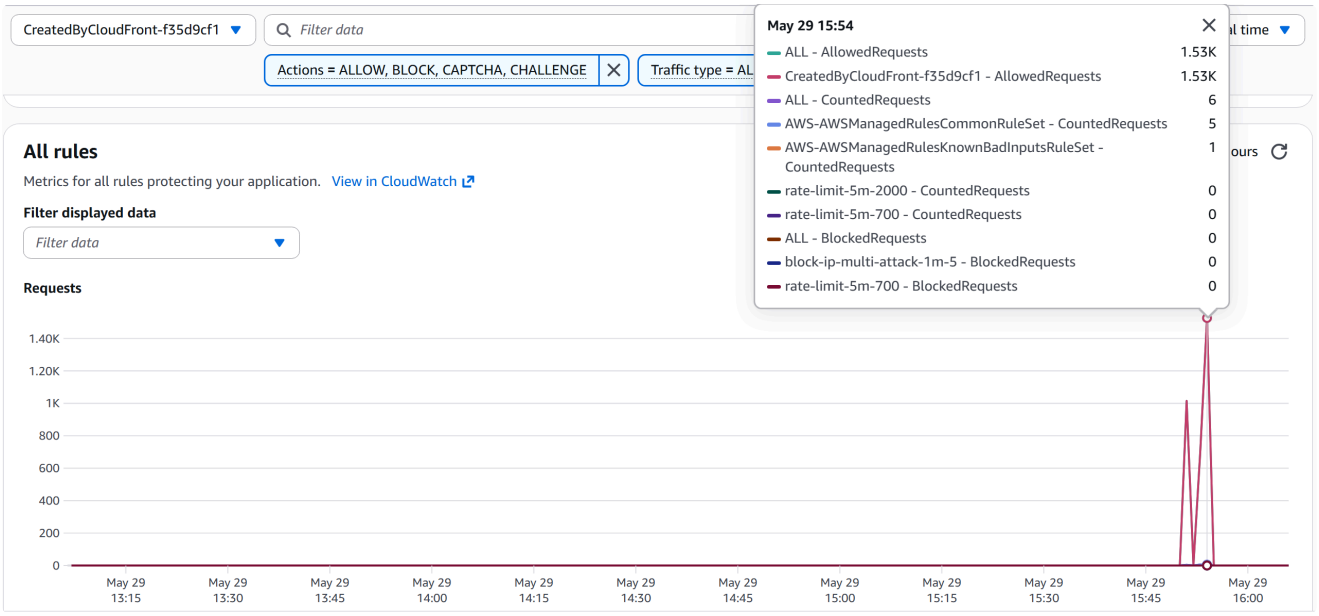


Sampled requests

client_ip varchar	country varchar	requests int64	unique_uris int64	matched_rules int64	uri_diversity_pct double
18.181.35.236	JP	13350	9126	2246	68.4
54.95.207.183	JP	4729	2982	2050	63.1
13.231.123.115	JP	4729	2981	1389	63.0
52.68.176.33	JP	4759	3008	1318	63.2
35.74.236.119	JP	4729	2982	1310	63.1
57.180.14.237	JP	4729	2982	1310	63.1
54.178.16.222	JP	4729	2981	1310	63.0

timestamp with time zone	varchar	int64
2026-05-29 04:21:08+00	ALLOW	56
2026-05-29 04:21:09+00	ALLOW	128
2026-05-29 04:21:10+00	ALLOW	124
2026-05-29 04:21:11+00	ALLOW	113
2026-05-29 04:21:12+00	ALLOW	113
2026-05-29 04:21:13+00	ALLOW	105
2026-05-29 04:21:14+00	ALLOW	124
2026-05-29 04:21:15+00	ALLOW	127
2026-05-29 04:21:16+00	ALLOW	88
2026-05-29 04:21:17+00	ALLOW	119
2026-05-29 04:21:18+00	ALLOW	106
2026-05-29 04:21:19+00	ALLOW	112
2026-05-29 04:21:20+00	ALLOW	118
2026-05-29 04:21:21+00	ALLOW	93
2026-05-29 04:21:22+00	ALLOW	127
2026-05-29 04:21:23+00	ALLOW	104
2026-05-29 04:21:24+00	ALLOW	113
2026-05-29 04:21:25+00	ALLOW	124
2026-05-29 04:21:26+00	ALLOW	83
2026-05-29 04:21:26+00	BLOCK	10
·	·	·
·	·	·
·	·	·
2026-05-29 04:22:26+00	BLOCK	145
2026-05-29 04:22:27+00	BLOCK	146
2026-05-29 04:22:28+00	BLOCK	148
2026-05-29 04:22:29+00	BLOCK	146
2026-05-29 04:22:30+00	BLOCK	149
2026-05-29 04:22:31+00	BLOCK	146
2026-05-29 04:22:32+00	BLOCK	147
2026-05-29 04:22:33+00	BLOCK	149
2026-05-29 04:22:34+00	BLOCK	145
2026-05-29 04:22:35+00	BLOCK	149
2026-05-29 04:22:36+00	BLOCK	149
2026-05-29 04:22:37+00	BLOCK	147
2026-05-29 04:22:38+00	BLOCK	148
2026-05-29 04:22:39+00	BLOCK	148
2026-05-29 04:22:40+00	BLOCK	149
2026-05-29 04:22:41+00	BLOCK	150
2026-05-29 04:22:42+00	BLOCK	149
2026-05-29 04:22:43+00	BLOCK	150
2026-05-29 04:22:44+00	BLOCK	144
2026-05-29 04:22:45+00	BLOCK	23

這是針對性攻擊之後的 nuclei



實際去看整個掃描事件的 log

```sql
SELECT
    date_trunc('minute', ts) AS minute,
    client_ip,
    count(*) AS total,
    count(*) FILTER (WHERE action = 'BLOCK') AS blocked,
    count(*) FILTER (WHERE len(nonTerminatingMatchingRules) > 0) AS matched
FROM waf
WHERE ts BETWEEN '2026-05-29 07:53:00+00' AND '2026-05-29 07:56:00+00'
GROUP BY 1, 2
HAVING total > 10
ORDER BY 1;
```

分兩次掃描，先找目標，再針對目標可能出現的漏洞掃描

minute timestamp with time zone	client_ip varchar	total int64	blocked int64	matched int64
2026-05-29 07:53:00+00	13.158.44.125	697	0	6
2026-05-29 07:54:00+00	13.158.44.125	1526	0	6

waf 基本上完全抓不到

sec timestamp with time zone	action varchar	cnt int64
2026-05-29 07:53:18+00	ALLOW	24
2026-05-29 07:53:19+00	ALLOW	202
2026-05-29 07:53:20+00	ALLOW	173
2026-05-29 07:53:21+00	ALLOW	134
2026-05-29 07:53:22+00	ALLOW	116
2026-05-29 07:53:23+00	ALLOW	48
2026-05-29 07:54:33+00	ALLOW	1
2026-05-29 07:54:34+00	ALLOW	1
2026-05-29 07:54:35+00	ALLOW	2
2026-05-29 07:54:36+00	ALLOW	2
2026-05-29 07:54:37+00	ALLOW	3
2026-05-29 07:54:38+00	ALLOW	9
2026-05-29 07:54:39+00	ALLOW	18
2026-05-29 07:54:40+00	ALLOW	21
2026-05-29 07:54:41+00	ALLOW	26
2026-05-29 07:54:42+00	ALLOW	34
2026-05-29 07:54:43+00	ALLOW	60
2026-05-29 07:54:44+00	ALLOW	90
2026-05-29 07:54:45+00	ALLOW	99
2026-05-29 07:54:46+00	ALLOW	92
2026-05-29 07:54:47+00	ALLOW	92
2026-05-29 07:54:48+00	ALLOW	90
2026-05-29 07:54:49+00	ALLOW	89
2026-05-29 07:54:50+00	ALLOW	106
2026-05-29 07:54:51+00	ALLOW	107
2026-05-29 07:54:52+00	ALLOW	104
2026-05-29 07:54:53+00	ALLOW	103
2026-05-29 07:54:54+00	ALLOW	98
2026-05-29 07:54:55+00	ALLOW	102
2026-05-29 07:54:56+00	ALLOW	96
2026-05-29 07:54:57+00	ALLOW	78
2026-05-29 07:54:58+00	ALLOW	3

簡單整理一下這次更新 nuclei 掃描方式之後出現的問題

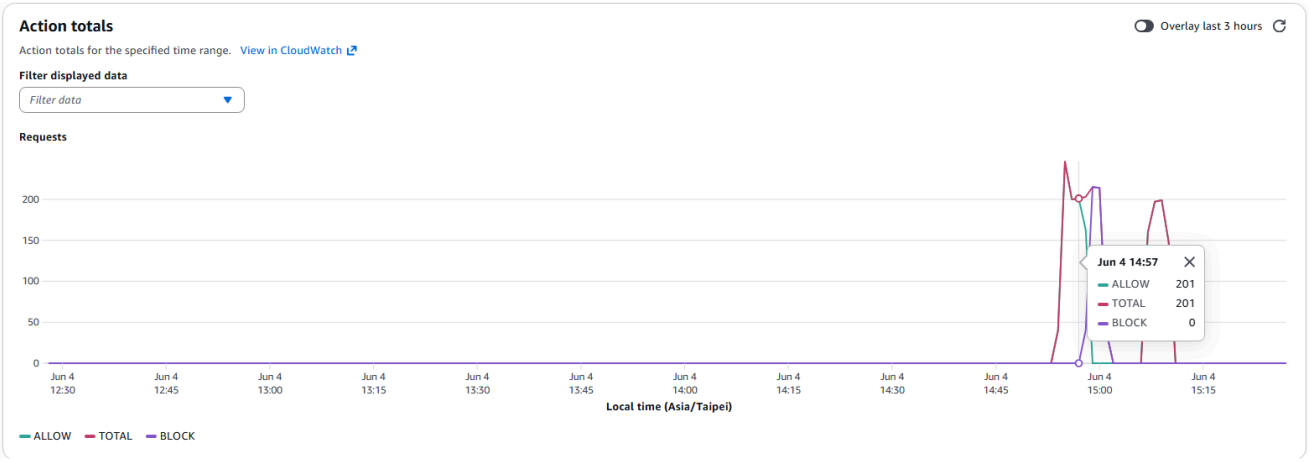
- 現在的 waf 偵測不到異常狀況，要一分鐘命中十次 default 的 rule 才會封鎖ip，但這次掃描被抓出來的 request 只有 6 條
- waf 一定會有延遲，就算發現是惡意攻擊也會需要 20 秒左右才真的封鎖 ip，但針對性的攻擊在 20 秒之前就可以打穿了

如果我現在沒有攻擊者的視角我要怎麼從 log 中找到攻擊的那條 query？

- 先看 query 的次數
- 列出可疑 IP
- 看他打了哪些 url 很多的話就很奇怪，可能是再掃描

20260604

寫了一個系統Jerry080401/normal_traffic來模擬正常流量



需要在正常流量中找到攻擊的 log 並且寫 rule 把它 block 掉

先建一個今天的 view

```
CREATE OR REPLACE VIEW waf_20260604 AS
      SELECT * FROM waf
      WHERE ts >= TIMESTAMPTZ '2026-06-04 00:00:00+08';
```

看一下有沒有出錯

```
SELECT count(*) AS total, min(ts) AS first, max(ts) AS last FROM waf_20260604;
```

total int64	first timestamp with time zone	last timestamp with time zone
2061	2026-06-04 06:54:56.037+00	2026-06-04 07:10:42.635+00

在看一下通過多少擋掉多少

```
SELECT action, count(*) FROM waf_20260604 GROUP BY 1 ORDER BY 2 DESC;
```

action varchar	count_star() int64
ALLOW	1554
BLOCK	507

接下來看一下正常的 log 會長怎樣

```
SELECT method, uri, count(*) AS hits FROM waf_20260604 GROUP BY 1,2 ORDER BY hits DESC LIMIT 20;
```

發現一個 endpoint 怪怪的，為甚麼只有一次

method varchar	uri varchar	hits int64
GET	/	874
GET	/api/v1/version	578
POST	/api/v1/login	155
GET	/flows	85
GET	/api/v1/folders/	77
GET	/api/v1/config	75
GET	/api/v1/flows/	71
GET	/api/v1/projects/	65
GET	/favicon.ico	40
GET	/manifest.json	40
POST	/api/v1/validate/code	1

先記下來，但感覺正常來說應該要看是誰打的流量?先看看 ua

```
SELECT ts, action, client_ip, method, uri,
             coalesce(list_filter(headers, h -> lower(h.name)='user-agent')[1].value, '(無 UA)') AS
ua
      FROM waf_20260604
      WHERE coalesce(list_filter(headers, h -> lower(h.name)='user-agent')[1].value, '') NOT ILIKE
'%Mozilla%'
      ORDER BY ts;
WARNING:
```

痾...好吧他們都一樣都有 ua

```
WARNING:
Deprecated lambda arrow (->) detected. Please transition to the new lambda syntax, i.e.., lambda x, i: x + i, before DuckDB's next release.
Use SET lambda_syntax='ENABLE_SINGLE_ARROW' to revert to the deprecated behavior.
For more information, see https://duckdb.org/docs/stable/sql/functions/lambda.html.
```

ts timestamp with time zone	action varchar	client_ip varchar	method varchar	uri varchar	ua varchar

0 rows

在找資料的時候發現 JA4+ Network Fingerprinting • FoxIO Blog 感覺可以用這個來判斷 (分群)

然後很簡單的

```sql
SELECT ja4Fingerprint, count(*) AS hits, count(DISTINCT uri) AS uris
      FROM waf_20260604
      GROUP BY 1
      ORDER BY hits ASC;
```

痾...好像很簡單就找出來了。如果在真實環境，我想到可以利用各種不同的分群分類找出那個異類然後加權，最後算每一條 log 的加權分數，分數越高代表越有可能是惡意流量。而且從 JA4 可以很快看到一些資訊，快速判斷對方是瀏覽器、腳本還是指令工具

ja4Fingerprint varchar	hits int64	uris int64
t13d251100_b78ed14e2fd0_ab7e3b40a677	3	2
t13d1812h1_85036bcba153_b26ce05bbdd6	2058	10

所以我叫 claude 寫了一個，依照

- 正常的 url 跟非正常
- 是不是已知的危險的 endpoint
- 看 waf 的 rule 有沒有叫
- 解析 ja4 看他是誰發的
  然後幫可能是攻擊的 log 加分，最後列出前20 就是最有可能是攻擊者的 Log

```sql
CREATE OR REPLACE VIEW scored AS
WITH base AS (
  SELECT *,
    coalesce(list_filter(headers, lambda h: lower(h.name)='user-agent')[1].value, '') AS ua,
    lower(uri || coalesce(query_string, '')) AS urlblob,
    count(*) OVER (PARTITION BY ja4Fingerprint) AS ja4_count
  FROM waf_20260604
),
feat AS (
  SELECT ts, client_ip, method, uri, action, ua, ja4Fingerprint, ja4_count,
    (CASE WHEN regexp_matches(urlblob,
        'oast\.site|interactsh|oastify|burpcollaborator|union\s+select|
<script|onerror=|javascript:|\.\./|etc/passwd|\$\{|/bin/sh|cmd=|exec\(') THEN 5 ELSE 0 END) AS
s_payload,
    (CASE WHEN uri NOT IN (
        '/', '/api/v1/version', '/api/v1/login', '/flows',
        '/api/v1/folders/', '/api/v1/config', '/api/v1/flows/',
        '/api/v1/projects/', '/favicon.ico', '/manifest.json'
      ) THEN 5 ELSE 0 END) AS s_offbaseline,
    (CASE WHEN uri = '/api/v1/validate/code'
            OR uri LIKE '/api/v1/build_public_tmp/%/flow'
            OR regexp_matches(lower(uri), '/\.env|/\.git|/admin|wp-login|/actuator|/console') THEN 2
ELSE 0 END) AS s_sensitive,
    (CASE WHEN len(labels) > 0 THEN 3 ELSE 0 END) AS s_label,
    (CASE WHEN len(terminatingRuleMatchDetails) > 0
            OR len(nonTerminatingMatchingRules) > 0 THEN 2 ELSE 0 END) AS s_wafmatch,
```

```
    (CASE WHEN ua NOT ILIKE '%Mozilla%' THEN 1 ELSE 0 END) AS s_ua,
    (CASE WHEN ja4Fingerprint IS NOT NULL AND ja4_count < 20 THEN 2 ELSE 0 END) AS s_ja4
  FROM base
)
SELECT *,
  s_payload + s_offbaseline + s_sensitive + s_label + s_wafmatch + s_ua + s_ja4 AS risk
FROM feat;
```

```
WITH long AS (
        UNPIVOT scored
        ON s_payload, s_offbaseline, s_sensitive, s_label, s_wafmatch, s_ua, s_ja4
        INTO NAME signal VALUE score
      ),
      ranked AS (
        SELECT signal, ts, client_ip, method, uri, action, score,
               row_number() OVER (PARTITION BY signal ORDER BY ts) AS rn
        FROM long
        WHERE score > 0
      )
      SELECT signal, rn, ts, client_ip, method, uri, action, score
      FROM ranked
      WHERE rn <= 20
      ORDER BY signal, rn;
```

結果長這樣，發現有一個 IP 很可疑 `3.112.132.45`

signal varchar	rn int64	ts timestamp with time zone	client_ip varchar	method varchar	uri varchar	action varchar	score int32
s_ja4	1	2026-06-04 06:57:20.608+00	3.112.132.45	POST	/api/v1/validate/code	ALLOW	2
s_ja4	2	2026-06-04 06:57:20.609+00	3.112.132.45	GET	/api/v1/version	ALLOW	2
s_ja4	3	2026-06-04 06:57:20.61+00	3.112.132.45	GET	/api/v1/version	ALLOW	2
s_offbaseline	1	2026-06-04 06:57:20.608+00	3.112.132.45	POST	/api/v1/validate/code	ALLOW	5
s_sensitive	1	2026-06-04 06:57:20.608+00	3.112.132.45	POST	/api/v1/validate/code	ALLOW	2

準備來分析 payload
拿到 ja4 跟 uri

```
waf D SELECT ts, method, uri, ja4Fingerprint
      FROM waf_20260604
      WHERE client_ip = '3.112.132.45'
      ORDER BY ts;
```

ts timestamp with time zone	method varchar	uri varchar	ja4Fingerprint varchar
2026-06-04 06:57:20.608+00	POST	/api/v1/validate/code	t13d251100_b78ed14e2fd0_ab7e3b40a677
2026-06-04 06:57:20.609+00	GET	/api/v1/version	t13d251100_b78ed14e2fd0_ab7e3b40a677
2026-06-04 06:57:20.61+00	GET	/api/v1/version	t13d251100_b78ed14e2fd0_ab7e3b40a677

```
{
  "Name": "Block-Nuclei-JA4-and-Langflow-RCE",
  "Priority": 10,
  "Action": { "Count": {} },
  "Statement": { "OrStatement": { "Statements": [

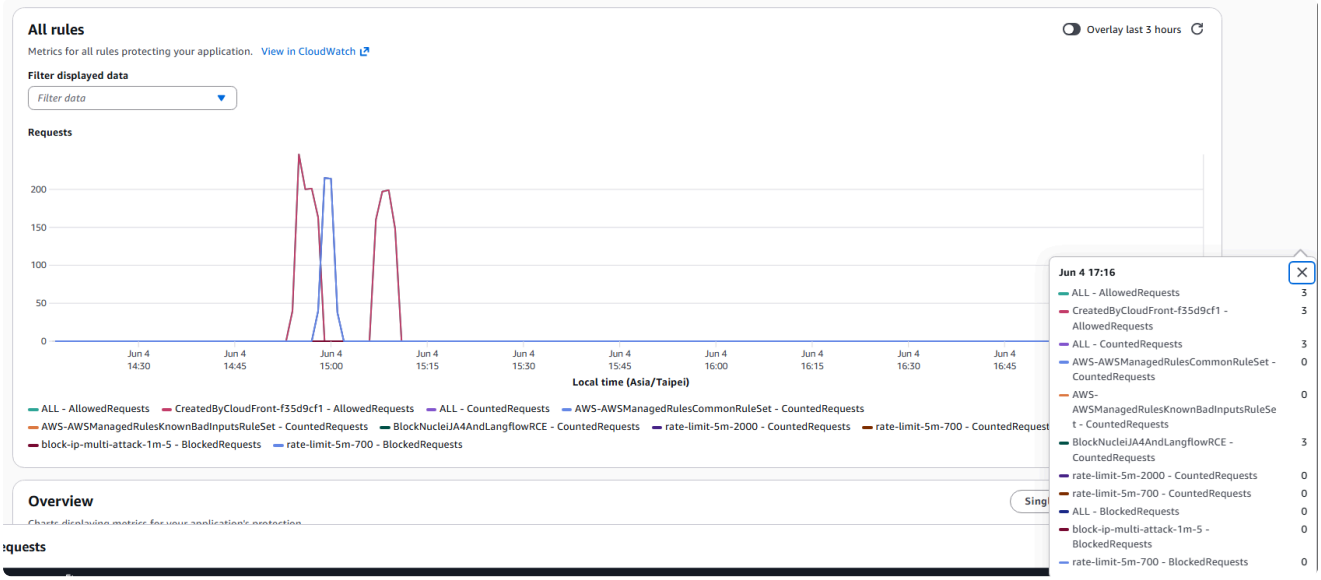
    { "ByteMatchStatement": {
        "FieldToMatch": { "JA4Fingerprint": { "FallbackBehavior": "NO_MATCH" } },
        "PositionalConstraint": "EXACTLY",
        "SearchString": "t13d251100_b78ed14e2fd0_ab7e3b40a677",
        "TextTransformations": [{ "Priority": 0, "Type": "NONE" }] } },

    { "AndStatement": { "Statements": [
        { "ByteMatchStatement": {
            "FieldToMatch": { "Method": {} },
            "PositionalConstraint": "EXACTLY", "SearchString": "POST",
            "TextTransformations": [{ "Priority": 0, "Type": "NONE" }] } },
```

```
        { "OrStatement": { "Statements": [
            { "ByteMatchStatement": {
                "FieldToMatch": { "UriPath": {} },
                "PositionalConstraint": "EXACTLY", "SearchString": "/api/v1/validate/code",
                "TextTransformations": [{ "Priority": 0, "Type": "URL_DECODE" }, { "Priority": 1,
"Type": "LOWERCASE" }] } },
            { "RegexMatchStatement": {
                "FieldToMatch": { "UriPath": {} },
                "RegexString": "^/api/v1/build_public_tmp/[^/]+/flow$",
                "TextTransformations": [{ "Priority": 0, "Type": "URL_DECODE" }, { "Priority": 1,
"Type": "LOWERCASE" }] } }
        ] } }
    ] } }

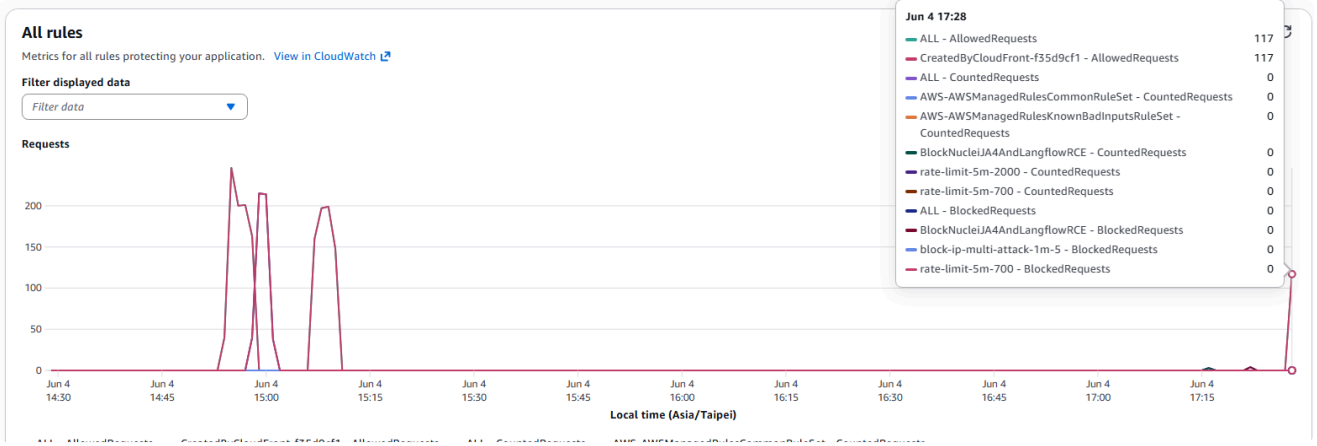
  ] } },
  "VisibilityConfig": { "SampledRequestsEnabled": true, "CloudWatchMetricsEnabled": true,
"MetricName": "BlockNucleiJA4AndLangflowRCE" }
}
```

確實成功了





正常的流量通過也不會有問題



小結

這幾天站在防守方，目標是用 WAF 擋下 nuclei 掃描，歷程大致是這樣。

1. 先觀察沒有 WAF 時的流量（一次 CVE 掃描約 4693 筆），加一條 rate-based rule 並調到 5 分鐘 700 筆。
2. 改成 block 後發現實際攔截量偏少，用 DuckDB 分析時間軸，確認 rate-based rule 有約 20~30 秒延遲（AWS 文件也這麼說），純靠 IP rate limit 擋不住針對性快攻。
3. 於是改用 label match，某個 IP 多次命中 AWS 內建 managed rules（admin-protection / known-bad-inputs / core-rule-set）就 block，封鎖時間動態、誤觸會自動恢復——大範圍掃描能攔到約 9K/13K，但兩段式的針對性掃描仍幾乎抓不到。
4. 最後進到在正常流量中抓出攻擊，用 JA4 指紋分群 + 自寫加權評分（off-baseline URL、敏感路徑、WAF label、JA4 稀有度…）把最可疑的 log 排到前面，揪出可疑 IP `3.112.132.45`，再寫出結合 JA4 指紋與 Langflow RCE 路徑的 block rule，實測能擋住攻擊又不影響正常流量。

參考資料

- Rate-based rule statement - AWS WAF
- Rate-based rule caveats - AWS WAF
- AWS Managed Rules rule groups
- Label match rule statement - AWS WAF
- JA4+ Network Fingerprinting（FoxIO-LLC/ja4）